

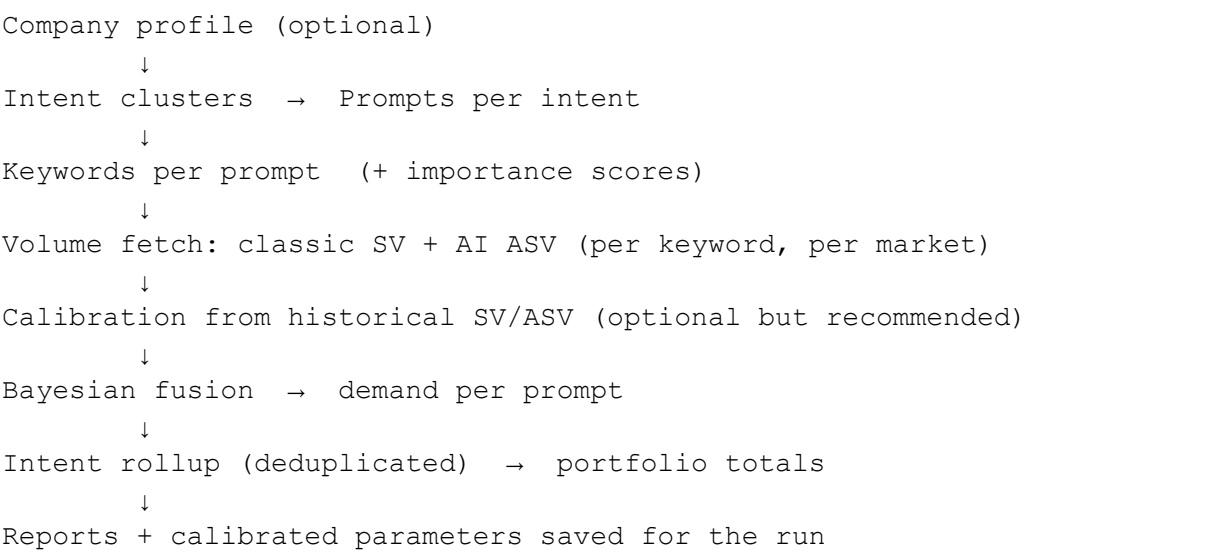
AI Demand Estimation — How It Works

What we need to start

Input	Role
Prompts	Real buyer questions (e.g. “best health plan for senior parents in India”)
Intent groups	Business themes to roll results up by (e.g. Family cover, OPD, Top-up)
Market	Country and language for volume data (e.g. India, English)
Company profile (optional)	Brand, industry, competitors — used when prompts are generated rather than supplied

Prompts can come from the customer, from Gravton, or be generated from a company profile using an LLM.

End-to-end process



Each run writes a timestamped folder under `runs/<run_id>/` with prompt estimates, keyword drill-downs, calibration output, and an executive summary.

Step 1 — Organize by buyer intent

Prompts are grouped into **intent clusters** — themes that reflect how buyers think (e.g. “Competitor alternatives”, “Family & senior coverage”, “Top-up plans”).

- If the customer already supplies prompts with intent labels, we use those directly.
- Otherwise, We get the prompts from prompt sythesis.

This grouping matters later: several calibration parameters are learned **per intent**, not only globally.

Step 2 — Extract keywords from each prompt

Each prompt is long and conversational; volume APIs need **short search phrases**. We extract several keywords per prompt and score how well each phrase represents the prompt.

Two extraction modes

Mode	How it works	When used
LLM (default)	An LLM proposes 2–6 Google-style search phrases grounded only in the prompt text. Phrases must be distinct angles, not minor rewrites of each other.	Recommended for production runs
N-gram	The prompt is split into 2- and 3-word phrases (stopwords removed); a semantic ranker scores each phrase against the full prompt.	Fallback or offline use

Importance scores

Every keyword gets an **importance score** between 0 and 1:

- The LLM only proposes candidate phrases; a **semantic ranker** scores how well each phrase matches the full prompt (same ranker used in n-gram mode).

- Invalid outputs are filtered out (e.g. phrases that are just a copy of the prompt, or off-topic additions not mentioned in the prompt).
- Higher score → that phrase is treated as a stronger signal when we later combine keyword-level demand into a prompt total.

Typical output per prompt: 5–12 keywords, ranked from most representative (often ~0.9+) to weaker tail phrases (~0.01–0.1).

What good keywords look like

- **Grounded:** only concepts that appear in the prompt (no invented competitors, products, or topics).
- **Search-shaped:** 2–4 words, the kind of query someone would type into Google.
- **Non-overlapping:** “health insurance india” and “health insurance policy” should not both appear unless they represent genuinely different search intent.

Step 3 — Fetch search volume (current snapshot)

For every unique keyword across all prompts, we query **DataForSEO** in the chosen market:

Sensor	Meaning	Used as
SV (classic search volume)	Traditional monthly Google search volume	Baseline demand scale
ASV (AI search volume)	Estimated volume in AI-powered search / AI mode	Direct AI demand signal

We also pull **CPC** and **competition** for each keyword — these feed calibration, not the raw volume read itself.

If a keyword returns no data, it is treated as zero volume for that run (conservative).

Step 4 — Calibrate the model from historical data

When calibration is enabled (`--with-calibration`), we pull **monthly history** for the same keywords (SV and ASV over recent months) and estimate parameters that describe how classic search relates to AI search **for this customer's keyword set and intent groups**.

This step runs **before** prompt-level estimation. Output is saved as `calibrated.json` in the run folder.

Parameters estimated from historical data

Parameter	What it means	Where it is learned	Historical inputs
ρ (rho) — AI share	What fraction of classic demand shows up as AI search for a keyword	Per keyword	Monthly SV vs ASV ratios, CPC, competition
η (eta) — AI uplift	Global multiplier: after accounting for ρ , how much AI volume exceeds what classic search alone would predict	Global (one value per run)	Residuals: $\log(\text{ASV}) - \log(\text{SV}) - \log(\rho)$ across keyword-months
SV prior (v, ω, σ_S)	How much typical keywords vary in classic volume, and how noisy monthly SV readings are	Per intent cluster (pooled fallback if too few keywords)	Matrix of monthly SV per keyword
ASV prior (μ, τ, σ_A)	Same for AI volume — typical level, spread across keywords, measurement noise	Per intent cluster	Matrix of monthly ASV per keyword
ρ coefficients ($\alpha_0, \alpha_1, \alpha_2$)	Logistic model linking CPC + competition to ρ	Global fit across keywords	Empirical ρ vs CPC/comp when enough keywords exist

How ρ is learned

1. Start with a default relationship: higher CPC and competition $\rightarrow$ lower AI share (commercial keywords behave differently).
2. For each keyword with history, compute an **empirical AI share**: average of $\text{ASV} \div (\eta \times \text{SV})$ across months.
3. Refit the logistic model so predicted ρ matches empirical ρ , using CPC and competition as features.
4. Apply a **floor** (minimum AI share $\approx 25\%$) so sparse or noisy data does not collapse estimates to zero.

How η is learned

1. For each keyword-month where both SV and ASV exist, compute an uplift residual:
“How much bigger is AI volume than classic volume, after removing the keyword’s AI share?”
2. The average of these residuals becomes **log(η)** — the global uplift factor.
3. Apply a **floor** (minimum $\eta \approx 1.3\times$) for stability.

How SV/ASV priors are learned (per intent)

For each intent cluster with enough keywords:

- **Within-keyword variance** over months → measurement noise (how much a single month’s reading jitters).
- **Between-keyword variance** (after removing noise) → how spread-out demand is across phrases in that intent.
- **Mean log-volume** → typical scale for keywords in that intent.

Intents with too few keywords inherit a **pooled** estimate from all keywords in the run.

Stability floors (always applied after calibration)

Floor	Purpose
Minimum ρ per keyword	Prevents AI share from going unrealistically low
Minimum η globally	Prevents under-estimating AI uplift when history is thin
Minimum ASV noise σ_A	Prevents over-trusting a single noisy ASV point in fusion

When calibration is **off**, fixed default priors are used instead (generic assumptions, less tailored to the category).

Step 5 — Estimate demand per prompt (Bayesian fusion)

For each prompt, we fuse SV and ASV across its keywords into one **monthly AI demand estimate** with a confidence range.

Per keyword

1. **Classic path:** Observed SV is combined with the calibrated SV prior → posterior estimate of “true” classic demand for that phrase.
2. **Coupling:** Classic demand is converted to an expected AI demand using that keyword’s ρ and the global η .
3. **AI path:** Observed ASV is treated as a noisy measurement of AI demand.
4. **Fusion:** The coupling expectation and the ASV observation are merged into a single posterior **AI demand** for the keyword (median, mean, and 90% interval).

Neither SV nor ASV is trusted alone — fusion down-weights whichever sensor is noisier for that intent.

Prompt total (fused)

Keywords are combined using **fusion weights** derived from importance scores:

- Higher importance → exponentially more weight (controlled by β , fusion sharpness).
- High β concentrates demand on the best-matching phrase; lower β spreads it across more keywords.

The reported **fused prompt volume** is this weighted blend.

A separate **linear** total (unweighted sum of keyword volumes) is also available as an upper-bound-style view when many weakly related keywords carry volume.

Step 6 — Roll up by intent group

Summing every prompt row would **double-count** the same search phrase appearing under multiple prompts in one intent. Intent totals use a different path:

1. **Union** all keywords across prompts in the intent; keep the **maximum** importance score when the same phrase appears twice.
2. **Semantic dedupe:** embed phrases and drop near-duplicates (e.g. “medicclaim family” vs “family medicclaim”) above a similarity threshold.
3. Run the same Bayesian fusion **once** on the deduplicated keyword set for that intent.

The numbers in [insights.md](#) (per-intent totals and 90% CIs) come from this deduplicated portfolio estimate — **not** a simple sum of prompt rows.

What the customer gets

Deliverable	What it shows
-------------	---------------

<code>prompt_estimates.csv</code>	Fused AI demand per question, uncertainty interval, intent group
<code>prompt_keyword_volumes.csv</code>	Drill-down: keywords per prompt, SV, ASV, fusion weights, keyword-level AI demand
<code>keyword_volumes.csv</code>	Master keyword list with fetched SV/ASV
<code>keyword_extractions.jsonl</code>	Keywords + importance scores extracted from each prompt
<code>calibrated.json</code>	All parameters learned from historical data (ρ , η , per-intent priors, sample counts)
<code>historical_sv_asv.json</code>	Raw monthly history used for calibration
<code>insights.md</code>	Executive summary: total demand and per-intent totals with confidence ranges

All outputs live in a timestamped run folder for sharing and comparison across runs.

Interpreting the numbers

- **Units:** AI demand units per month — comparable within a run, anchored to search data for the chosen market.
- **Fused vs linear:** Fused focuses on phrases that best represent the prompt; linear sums all keyword volumes without that weighting.
- **Low volumes on long-tail prompts:** Specific or niche questions often have little SV/ASV on extracted phrases; fused estimates stay conservative by design.
- **Intent vs prompt totals:** Use intent totals for portfolio planning; use prompt rows for question-level prioritization.

One-line summary

Prompts → grounded keywords with importance scores → classic + AI volume → historical calibration (ρ , η , per-intent priors) → Bayesian fusion → per-prompt and deduplicated per-intent demand report.